Questioning Agreement: Selecting Reliable Mathematical Answers with Complementary Verifiers

DASE7506 MP2 — experimental report

Abstract

When four generated solutions disagree, a fluent explanation or a majority answer need not be correct. I investigate whether a compact selector can combine answer agreement, candidate representations and question–solution interaction to select a correct final answer. My model combines a normalized baseline with a group, kernel and text ensemble. I proposed a 25% baseline / 75% ensemble ratio before testing that ratio; it obtained **391/467 (83.72591006423983%)** on the official test scorer. The normalized baseline obtained 383/467. On validation, however, the same blend obtained 739/798 versus 752/798 for the baseline. Three training cross-validation splits show small positive aggregate gains but inconsistent fold-level changes. I examine disagreement cases and rejected hypotheses rather than treating a higher score as proof of sound reasoning. The score is below 84%, which requires at least 393 correct answers.

1. Research question and motivation

My central question is: **can comparing complementary evidence identify a correct answer when answer voting and a learned selector disagree?** A wrong solution may be persuasive, and several candidates may repeat the same mistake. Conversely, an imperfect derivation can accidentally end with the right number. The course scorer evaluates the final number, not whether the derivation is valid [1]. A selector can therefore achieve a high score without being a sound step-by-step mathematical verifier.

I pursued these questions rather than committing to a single architecture: What information is added by considering other candidates? Does combining individually imperfect scorers help? Are failures explained by superficial features, and do improvements persist across folds? Course lectures on verifiers, self-consistency and reward or scoring failures motivated my investigation [2]. They are conceptual inspiration, not a claim that my system performs reinforcement learning or uses an LLM to verify solutions.

2. Data and evaluation protocol

The supplied GSM8K-derived bank contains 4,755 training, 798 validation and 467 test questions. Each has four candidate traces and one fixed 1,024-dimensional representation per candidate. I use the supplied parser and independent scorer: a selection is correct when its parsed final numeric answer equals the reference; multiple candidates in one question may be correct. I did not modify the data, parser or scorer. Inference chooses one supplied candidate, uses no reference answer or ID-based lookup, and makes no generation or

external API call.

I fit model parameters and transformation statistics on training data. My training-only diagnostic splits are by question; I use validation for development and comparison. **Disagreement questions** have at least two distinct parsable answers. Training contains 2,114 such questions, validation 355, and every test group has at least two distinct valid parsed answers. Reporting the disagreement subset helps separate easy agreement cases from the task of resolving conflicting answers, but it does not make the validation and test distributions identical.

I used validation repeatedly during development. I had seen aggregate results from earlier models before proposing the 25/75 ratio, and I froze each reported configuration before scoring that configuration. After evaluating the 25/75 model, I retained it as a reference for further analysis.

3. Method

3.1 Baselines

I use a fixed-seed random selector and answer voting as simple reference points. Voting groups equivalent parsed numbers and selects a candidate from the largest answer group. My stronger normalized baseline uses the supplied trace representations, learned candidate scoring and candidate-relative/answer-group features; it also normalizes supported written quantities in a copy of the input text. It retains the official parser for choosing scorable candidates. Its existing numerical features are part of the baseline, not a new feature introduced by this experiment. No normalization is guaranteed to resolve every linguistic ambiguity.

3.2 Complementary ensemble

I used an AI-assisted summary of material I found online to investigate three complementary components. I developed the implementation from that summary, rather than copying an available implementation or checkpoint; the original detailed source list was not retained (Section 8). The three components are:

1. **Group scorer (A):** a LambdaRank model compares groups of candidates with the same parsed answer. Its inputs include candidate-level pairwise-linear, LambdaRank and TF-IDF scores, projected candidate embeddings, and 65 scalar formatting, equation and answer-group attributes. Out-of-fold scores are used when fitting the group layer.
2. **Kernel scorer (K):** a pairwise logistic ranker uses a Nyström approximation to an RBF kernel with 512 landmarks. It provides nonlinear distinctions over candidate representations and scalar features.
3. **Text interaction scorer (C):** a pairwise ranker uses TF-IDF features from candidate traces and interactions between projected question and solution text. Thus the ensemble does use question text; fixed trace embeddings are not its only input.

Scores are centered across parsable candidates within each question and divided by their standard deviation, with a 0.1 floor. First, standardized A and K are combined at 25% and 75%, then standardized again. This intermediate score is

combined at 75% with standardized C at 25%. Call the resulting two-stage score `P`. The investigated final score is

`0.25 × standardized baseline + 0.75 × standardized P`.

I proposed the 25/75 outer ratio and fixed it before testing that ratio. I do not claim that the individual A, K or C scorers are each better than the baseline. Candidate answers that the official parser cannot score are masked when a parsable candidate exists. My implementation returns a candidate index, not a calibrated probability or a proof of reasoning.

4. Experiments and results

4.1 Baselines and fixed-ratio comparisons

I compare the methods on the same supplied validation and test banks using the official scoring protocol. The 25/75 and 75/25 ensemble rows use the same `peer–bottom` checkpoint; the 15/85 and 50/50 rows use fixed reweightings of that checkpoint. The 100/0 row is the normalized baseline. I froze the 25/75 model before scoring it, and I froze the 15/85 and 50/50 configurations before their later test comparison.

Method / baseline : ensemble	Validation correct / 798	Validation disagreement / 355	Test correct / 467
Random, seed 2026	592	—	Not measured
Answer voting	688	245	Not measured
Normalized baseline, 100 : 0	752	309	383 (82.01%)
Ensemble, 75 : 25	750	307	383 (82.01%)
Ensemble, 50 : 50	744	301	383 (82.01%)
Ensemble, 25 : 75	739	296	**391 (83.73%)**
Ensemble, 15 : 85	738	295	389 (83.30%)

This comparison is an inference-time ablation of the outer blend, not retraining each model without a component. On this validation set the baseline is stronger; on this public test bank the investigated 25/75 model has eight more correct answers. These results illustrate that validation ranking did not predict the observed test ranking. They do not prove which model will perform best on unseen data, establish statistical significance, or demonstrate that any particular inner component caused the difference. Equal aggregate totals do not imply the same questions were answered correctly.

4.2 Matched training-only cross-validation

For seeds 6813, 7253 and 7507, I compared the 25/75 blend with the normalized baseline using matched question-level outer folds and nested fitting for the relevant component scores. On the 2,114 disagreement questions:

Seed	Baseline	25/75 blend	Difference
6813	1,704	1,716	+12
7253	1,731	1,734	+3
7507	1,721	1,724	+3

The nine outer-fold changes were `[6, 10, −4]`, `[−6, −1, 10]`, and `[−2, 4, 1]`. An exploratory rule set before examining these results required all three seed-level gains and at least six nonnegative fold gains. Only five of nine folds were nonnegative, so the blend did **not** meet that stability rule. These seeds repartition the same data and are not independent test sets; earlier development also reused the training data.

For seed 6813, the group, kernel and text scorers alone obtained 1,678, 1,700 and 1,703 disagreement questions respectively, versus 1,704 for the baseline. Their combined effect therefore cannot be explained simply by saying that each component was stronger. Complementarity is plausible, but these counts alone do not prove a causal mechanism.

5. Failure analysis and rejected hypotheses

I examined the seed-6813 held-out training predictions and found that the 25/75 blend missed 398 of 2,114 disagreement questions. Of those 398 errors, 315 questions had exactly one correct candidate; in 176 errors the selected answer appeared only once. Answer voting selected correctly on 1,413 disagreement questions, compared with 1,716 for the blend. The vote comparison supports using more than raw agreement, but the error counts do not identify a single failure mechanism.

On the reused validation disagreement subset, the baseline and 25/75 blend selected different final numbers on 20 questions. The baseline alone was correct on 16, the blend alone on three, and both were wrong on one. This paired breakdown is more informative than aggregate accuracy, but it remains exploratory because validation was repeatedly inspected.

I also investigated whether missing numeric vocabulary was a major bottleneck. A numeric-character backoff was added to the text ranker and compared on the same three training splits. Both the original blend and the backoff variant obtained 1,716, 1,734 and 1,724 correct disagreement questions respectively. The added channel changed some scores, but did not change paired correctness in these evaluations; it was not adopted. This negative result matters: a plausible hypothesis about unseen numeric tokens was not supported by this particular intervention. It does not prove that all number representation methods are useless.

The final-answer metric also limits what equation checks can establish. Some candidate equations can be misinterpreted by a simple checker, and a correct final answer can coexist with an invalid intermediate derivation. A conservative separate equation diagnostic did not justify a hard veto. Similarly, a temperature applied to a single candidate-score vector cannot change its argmax; a claim of improved accuracy from calibration alone would require an additional decision

mechanism and separate evaluation.

6. Efficiency and reproducibility

The reported 25/75 `peer-bottom` checkpoint contains **10,235,918 uncompressed bytes** of arrays, approximately 9.76 MiB, before counting any small configuration constants; the asset limit is 16 MiB. On an Apple M1 with four-thread limits, one fresh test prediction process plus independent scoring took **8.96 seconds** wall time including startup. The normalized baseline's recorded corresponding local time was **5.57 seconds**, with 607,272 checkpoint bytes. Thus the ensemble's observed eight-question test difference came with greater storage and local compute, not a free improvement. The specified Xeon Platinum 8457C was not measured, so reference-machine time compliance remains unverified.

The saved `peer-bottom` training comparison took approximately 1,045 seconds locally; the later three-seed diagnostic took approximately 712 seconds. These are individual recorded experiments, not the total cost of all development or a precise standalone final-training time. The final exported checkpoint uses training seed 7507. The recorded environment was Python 3.12.14, NumPy 2.5.3, SciPy 1.18.1, scikit-learn 1.9.1 and LightGBM 4.7.0. On the recorded macOS environment LightGBM also required an OpenMP runtime.

From `code/`, install `requirements-experiments.txt`, run `python check_data.py`, and set `OMP_NUM_THREADS`, `OPENBLAS_NUM_THREADS`, `MKL_NUM_THREADS` and `VECLIB_MAXIMUM_THREADS` to 4. Reproduction uses the frozen method and checkpoint, with no retraining:

```
python predict.py --method submission_peer_75.py --checkpoint runs/
peer-test/peer-bottom.npz \
  --split test --output runs/peer-test/reproduced-predictions.json
python score_predictions.py --split test \
  --predictions runs/peer-test/reproduced-predictions.json \
  --output runs/peer-test/reproduced-score.json
```

The checkpoint SHA-256 is `a174829d7952d347f21e1538ae612dfbb2ec30a9350e467e3cf2d81fe1d54529`. The pre-test freeze, inference dependency hashes, official score and resource record are in `code/runs/peer-test/`; fixed-ratio comparison records are in `code/runs/fixed-weight-comparison/`. Reproduction requires the matching source version and checkpoint, not cached test predictions. A reference-free reconstruction check passed on validation inputs and synthetic permutations; this is not a measurement of performance on the instructor's separate audit bank.

For a training reconstruction, `python export_peer.py` fits and exports the ensemble from the training split with seed 7507, using the existing training-derived `runs/numeric/selected.npz` as the old branch. Run it in a fresh copy without an existing `runs/peer-test/` directory: it intentionally refuses to overwrite that directory. The old-branch training procedure and the diagnostic commands are documented in `code/README.md`. This is a training reconstruction, whereas the frozen-checkpoint scoring commands above are the path for reproducing the reported result.

7. Conclusion

I progressed by proposing a combination, testing whether it changed outcomes, inspecting disagreement cases, and rejecting a numeric-backoff idea that did not improve the matched training comparisons. My proposed 25/75 model achieved 391/467 on the public test bank, but validation favored the baseline and fold gains were not uniformly stable. I do not simplify these results into “the ensemble always works” or “the ensemble is useless.” My remaining questions are whether the gain transfers to a genuinely unseen audit bank, which evidence resolves the hardest disagreements, and whether a verifier can distinguish genuinely sound reasoning from a lucky final number.

8. Attribution and substantive AI assistance

I defined the research goal, proposed the 25/75 outer blend, requested repeated comparisons, and pursued the reasons behind errors rather than stopping at aggregate scores. I used GitHub Copilot substantively for code generation and debugging, experimental execution, analysis, and drafting. I also used another AI to summarize material I found while researching online; that summary inspired my A/K/C direction. The summary did not retain a verifiable bibliography or the original full implementation, so I do not attribute it to a specific author, paper, or implementation. I will add the underlying sources if I recover them. My system is an independent approximation, not a verified reproduction of that summary. I am responsible for reviewing the code, claims, disclosures and submission.

[1] DASE7506 MP2, *Selecting Reliable LLM Reasoning*, supplied course guide and code README, 2026.

[2] DASE7506, course lectures 4–7, supplied lecture materials, 2026.

[3] GSM8K data attribution and license, `code/data/NOTICE.txt` and `code/data/LICENSE.txt`; candidate solutions and representations were prepared using Qwen3-0.6B as described in the supplied data manifest.